Graph

Data structure

Why Graph needed?

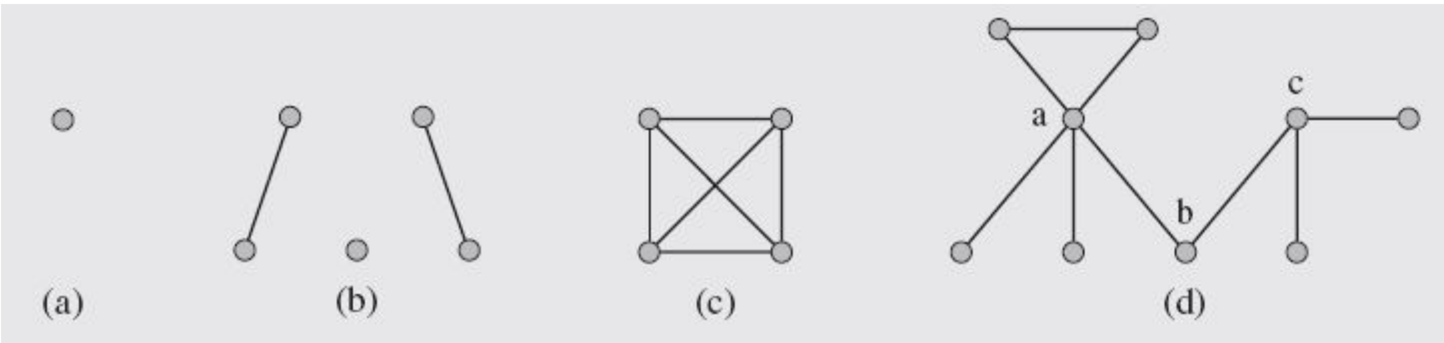
- Any time there is a set of objects and there is some sort of "connection" or "relationship" or "interaction" between pairs of objects, a graph is a good way to model this. Some examples are
  - Connecting with friends on social media
  - To find a route based on shortest route.
  - Transportation networks, e.g. ,roads
  - Google, to search for WebPages, where pages on the internet are linked to each other by hyperlinks
  - Page Rank
    Network Traffic flow
  - Minimum spanning tree etc

Graph

- A *graph G = (V, E) consists of a finite set of vertices V (or nodes) and E, a binary relation on V called edges. E is a set of pairs from V.*

- A graph is a collection of vertices (or nodes) and the connections between them.

Simple Graphs examples

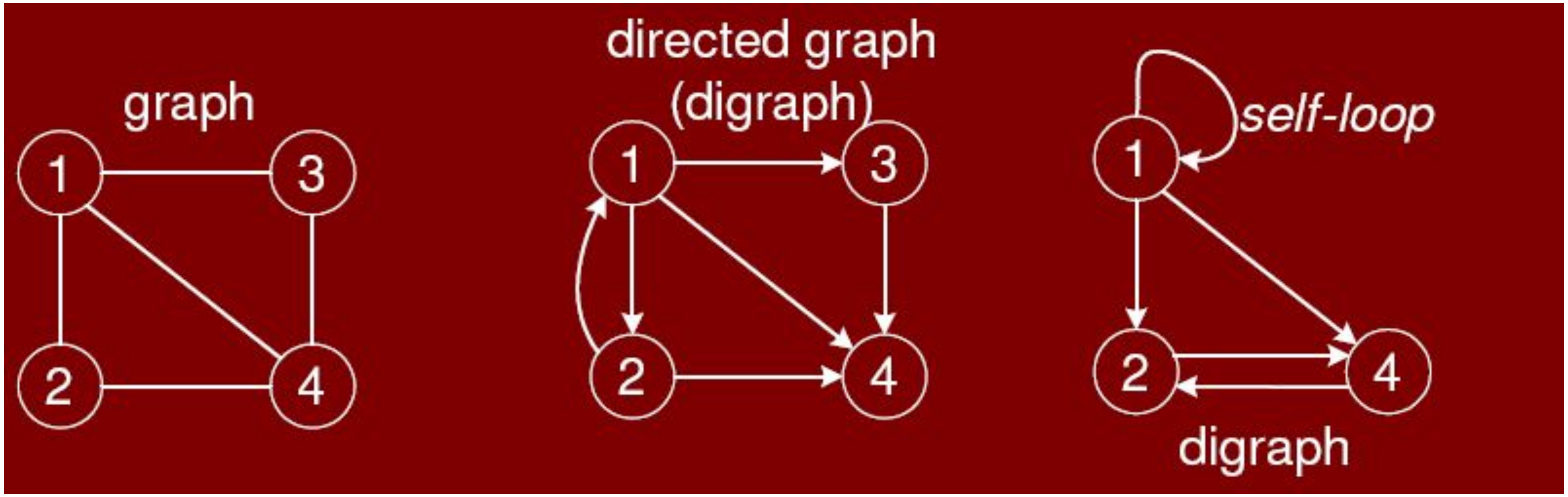


The number of vertices and edges is denoted by |V| and |E |, respectively.

For graph (a) |V|=1, |E|=0
For graph (b) |V|=5, |E|=2
For graph (c) |V|=4, |E|=6

Directed and undirected

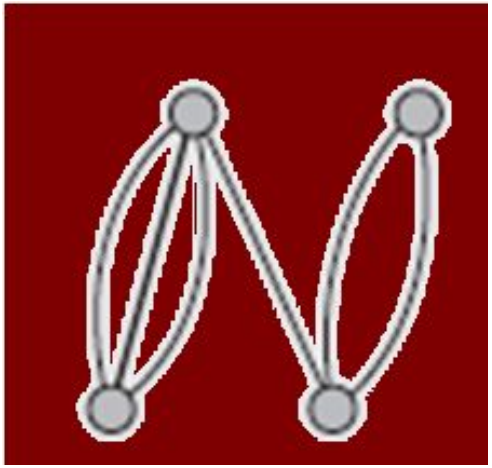
- E is a set of pairs from V. If a pair is **ordered**, we have a **directed** graph. For **unordered** pair, we have an **undirected** graph.
- In directed graph edges (2,4)≠(4,2)

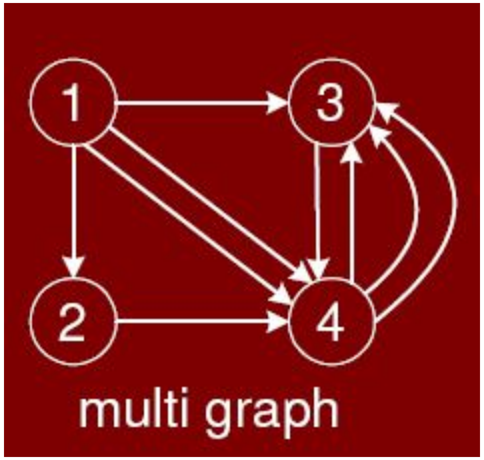


Note: In simple graph, self loop is not allowed.
A graph allows the self loop is known as pseudograph.

Multi graph

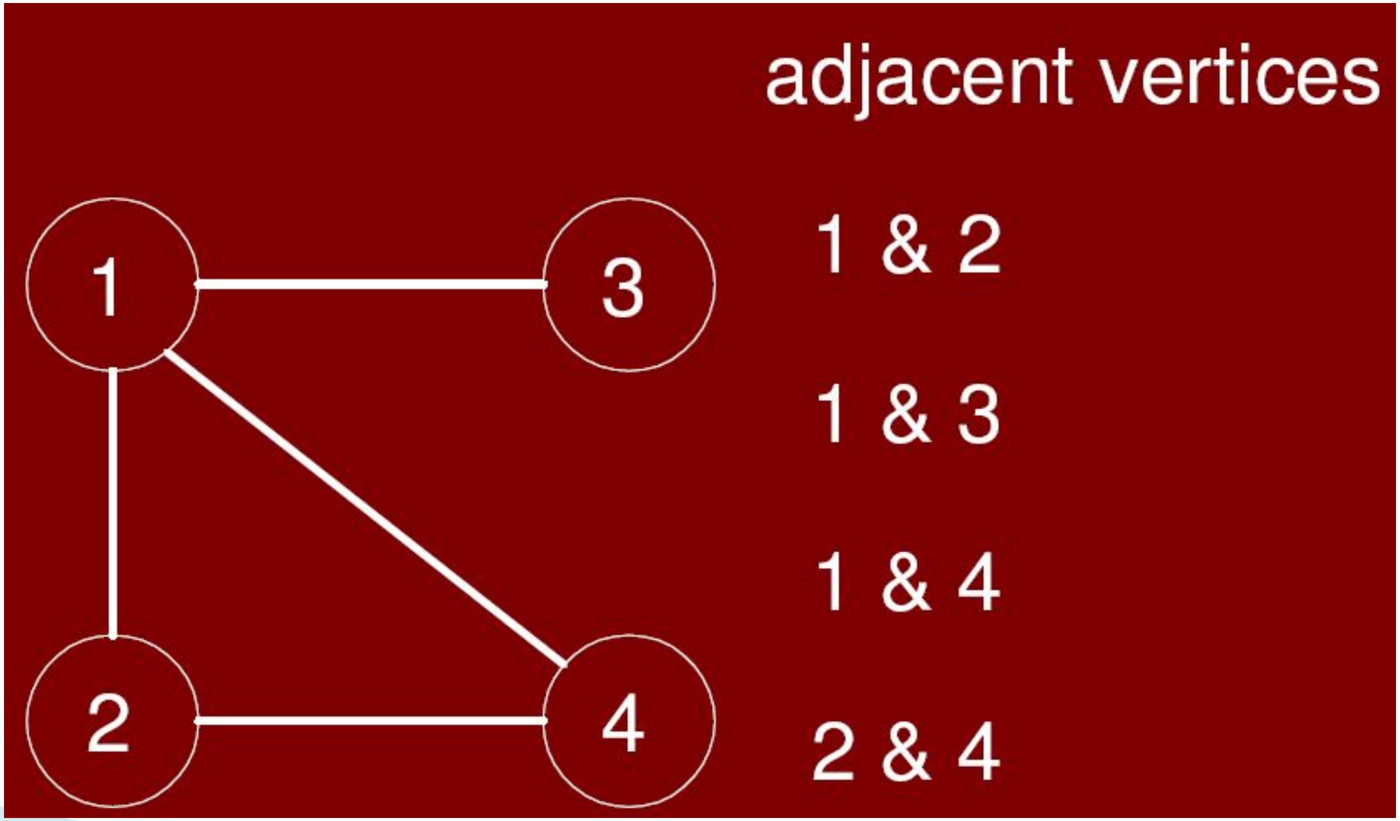
- A *multigraph* is a graph in which two vertices can be joined by multiple edges.





Adjacent Vertices

- A vertex w is *adjacent to vertex v* if there is an edge from v to w.

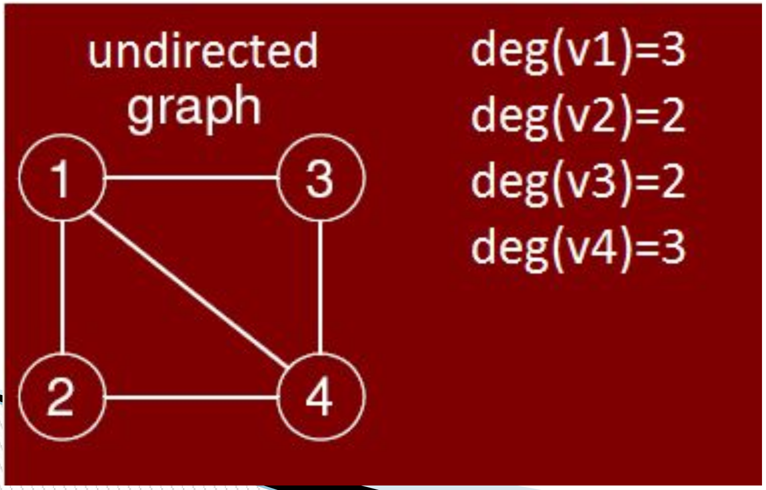


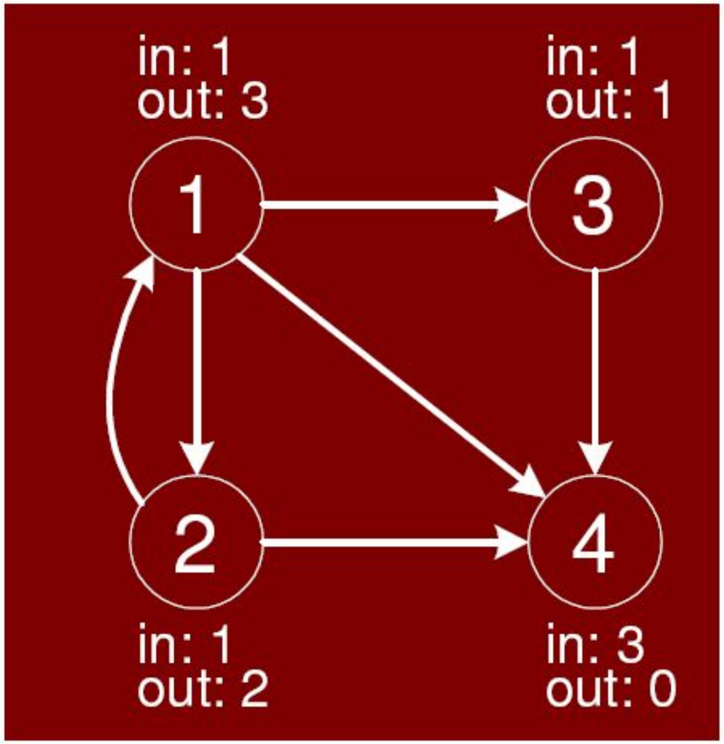
Incident on vertices

- In an undirected graph, we say that an edge is *incident on a vertex* if the vertex is an endpoint of the edge.

Degree of a Vertex

- The number of edges coming out of a vertex is called the ***out-degree*** of that vertex.
- Number of edges coming in is the ***in-degree.***
- In an undirected graph, we just talk of degree of a vertex.





In and out degrees of vertices of a graph

Hand Shaking lema

- For a directed digraph G = (V, E),

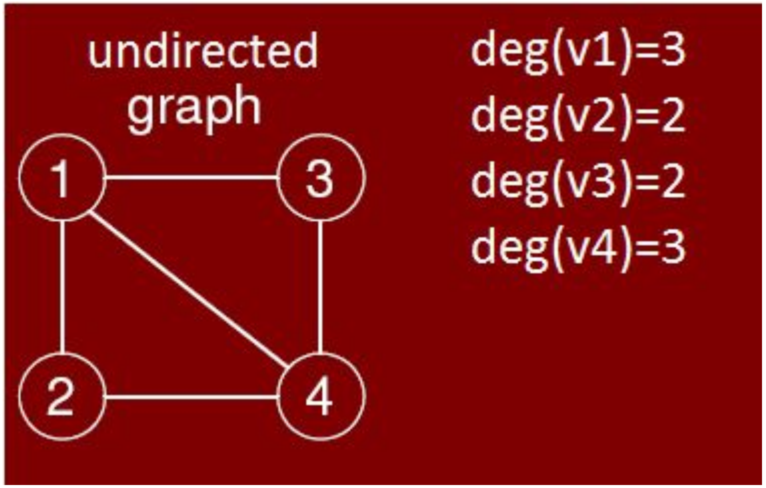
$$\sum_{v \in V} in\text{-}degree(v) = \sum_{v \in V} out\text{-}degree(v) = |E|$$

- For an undirected digraph G = (V, E),

$$\sum_{v \in V} degree(v) = 2|E|$$

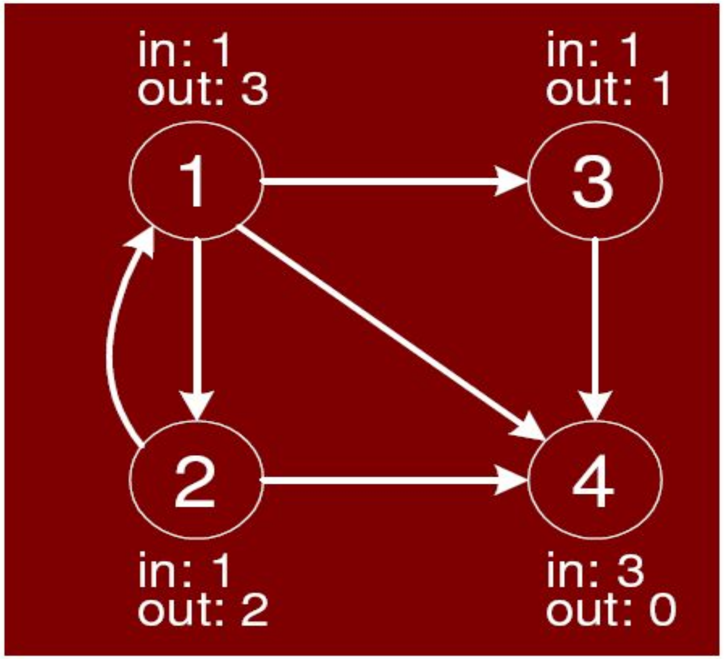
Example(undirected)

- **Undirected Graph**
  - Degree V= deg(v1)+ deg(v2)+ deg(v3)+ deg(v4)
  - =3+2+2+3=10=2(5)=2|E|
  - Total no. of edges=|E|=5



Example(directed)

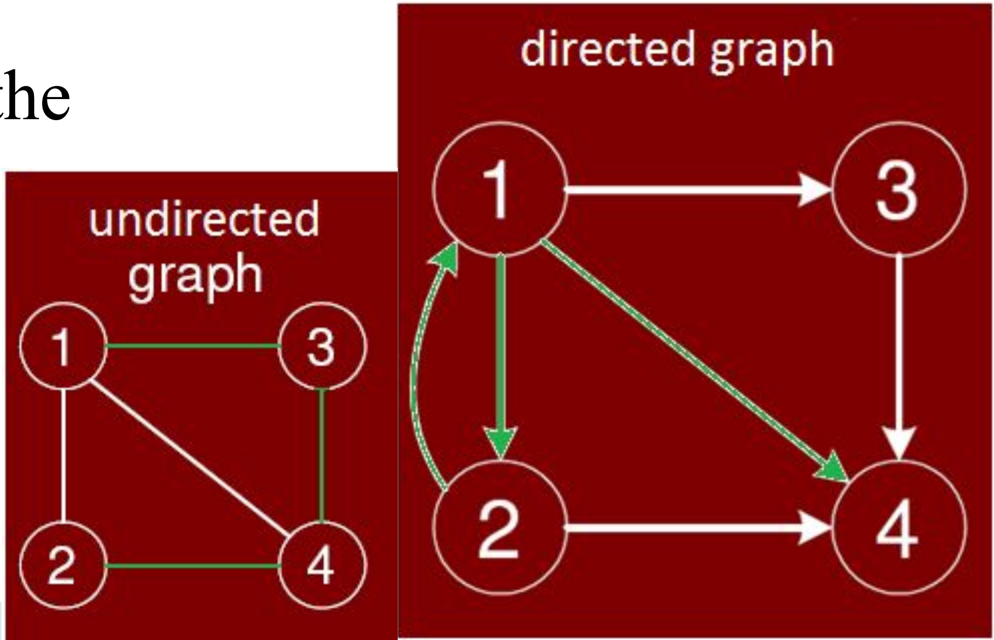
- **Directed Graph**
  - Indegree V= indeg(v1)+ indeg(v2)+ indeg(v3)+ deg(v4)
  - =1+1+3+1=6=|E|
  - Outdegree V= outdeg(v1)+ outdeg(v2)
    + outdeg(v3)+ outdeg(v4)
    =3+2+0+1=6=|E|



In and out degrees of vertices of a graph

Path

- A path in a directed graph is a sequence of vertices(v_1, v_2,……, v_k) such that (v_{i-1}, v_i) is an edge for i=1,2,3….k.
- Example of path,
  - In directed graph:1,2,1,4
  - In undirected graph:1,3,4,2
- The length of the path is the number of edges, k.
- Length of path
  - Directed graph=3
  - Undirected graph=3

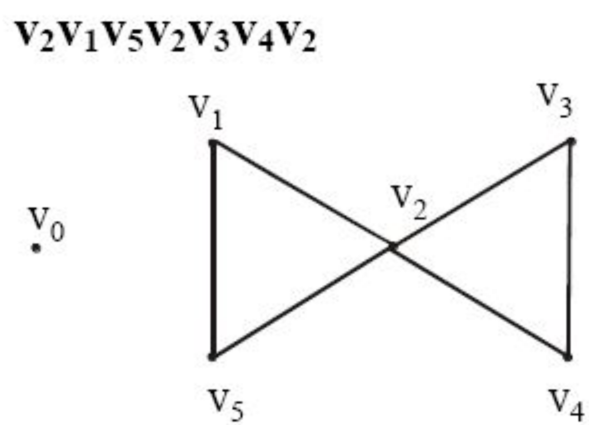


Simple path

- A **simple path is a** path such that all vertices are distinct.
- No edge is repeating in it.
- No vertex is repeating in it.
- Special case of simple path when start and end of vertex are same then it will be **cycle**.

Circuit

- If $v_1 = v_n$ and no edge is repeated, then the path is called a circuit
- In this graph, vertex v2 is repeated and the graph starts and end at the same vertex (i.e. at v2) and no edge is repeated, hence the above graph is a circuit



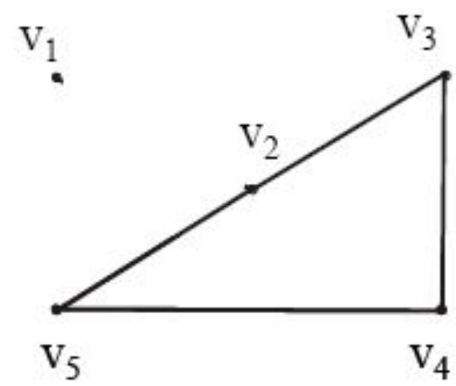
Summary

	Repeated Edge	Repeated Vertex	Starts and Ends at Same Point
circuit	no	Allowed	yes
cycle	no	first and last only	yes
path	no	Allowed	allowed
simple path	no	no	No

Cycle

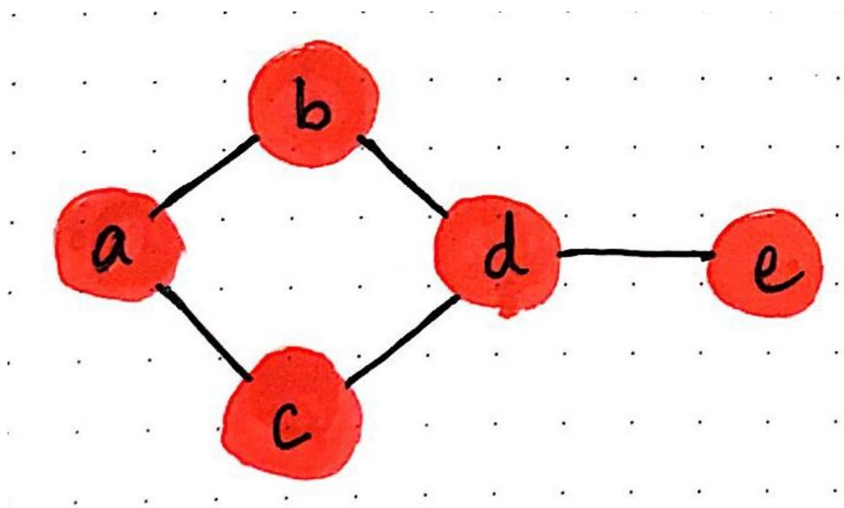
- If all vertices in a circuit are different except start and end, then it is called a *cycle.*
- In cycle, neither repeating edge is allowed nor repeating vertices is allowed. Starting and ending will be same.

$v_2 v_3 v_4 v_5 v_2$



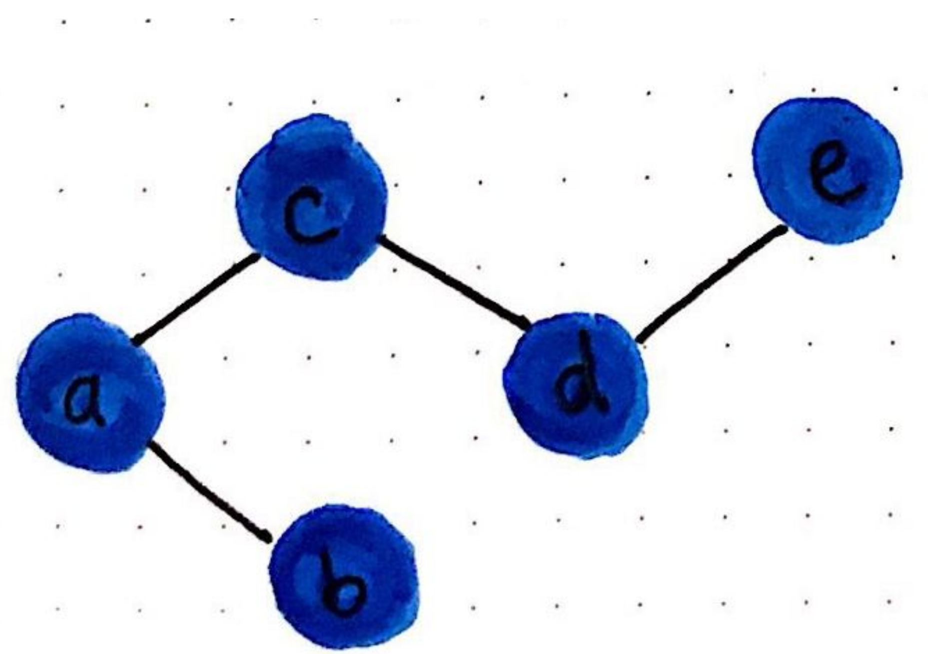
Cycle Graph

- A graph with at least one cycle is known as a cyclic graph.



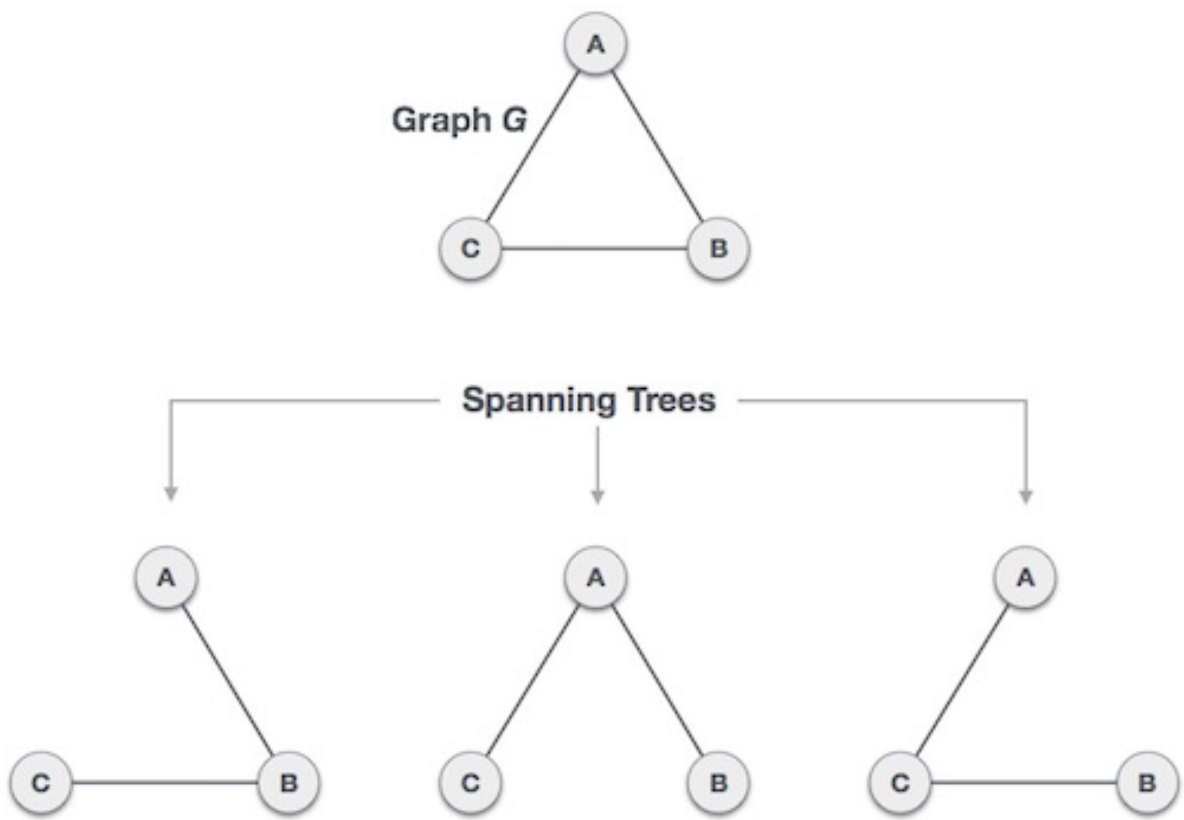
Acyclic graph

- A graph with no cycle in it is known as an acyclic graph



Spanning tree

- A **spanning tree** is a subset of Graph G, which has all the vertices covered with minimum possible number of edges.

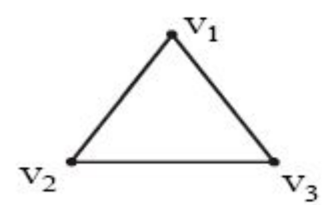


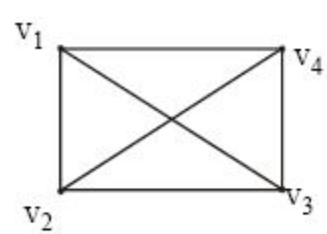
Complete Graph

- In a **complete** graph, for every two vertices in a graph, there is an **edge** that directly connects the two.
- A complete graph on n vertices is a simple graph in which each vertex is connected to every other vertex

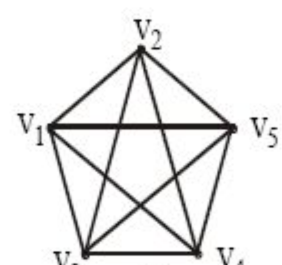
K_1

K_2





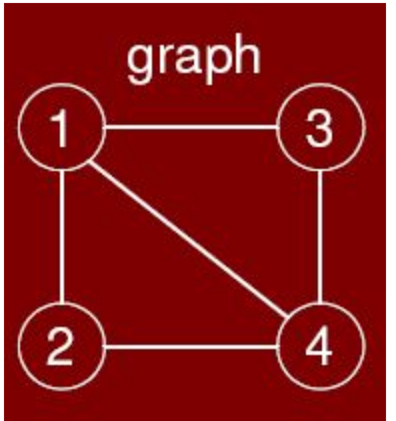
K_4



K_5

Connected graph

- A **connected graph** is any graph where there's a **path** between every pair of vertices in the graph.
- There is a path from 2 to 3 but does not have directed edge. so it is connected but not complete.
- A complete graph is always connected but connected graph may not be complete.
- If graph is directed and connected then **strongly connected**
- If undirected then **weakly connected**



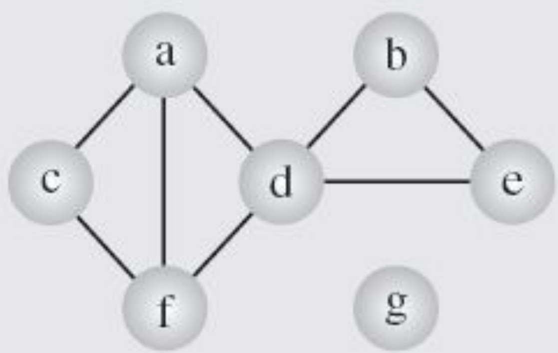
Graph Representation

- There are two ways of representing graphs:

1. Adjacency matrix
2. Adjacency list

Adjacency List

- A simple representation is given by an *adjacency list,* which specifies all vertices adjacent to each vertex of the graph.
- Storage space O(|V|+|E|) or O(V+2E)



(a)

a	c	d	f	
b	d	e		
c	a	f		
d	a	b	e	f
e	b	d		
f	a	c	d	
g				

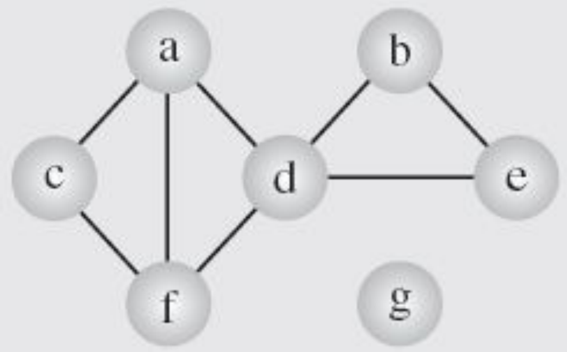
(b)

Adjacency Matrix

- An *adjacency matrix* of graph G = (V,E) is a binary |V| × |V| matrix such that each entry of this matrix.

$$a_{ij} = \begin{cases} 1 & \text{if there exists an edge} (v_i v_j) \\ 0 & \text{otherwise} \end{cases}$$

Note:
Storage space
O(|V|x |V|)



(a)

	a	b	c	d	e	f	g
a	0	0	1	1	0	1	0
b	0	0	0	1	1	0	0
c	1	0	0	0	0	1	0
d	1	1	0	0	1	1	0
e	0	1	0	1	0	0	0
f	1	0	1	1	0	0	0
g	0	0	0	0	0	0	0

(d)